

DA2304: Introduction to Computer Systems

Assignment 3 Report

Name: M. Sai Teja
Roll Number: DA24B031

May 20, 2026

Part 1: Design Justifications

Filter Selection

To test the `Conv` class, I implemented a standard 3×3 kernel:

$$F = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

I chose this kernel because it provides a reliable, high-contrast output that makes it easy to verify the sliding window logic. By initializing a 5×5 input matrix with a distinct vertical boundary (left side 0s, right side 10s), the convolution operation generates a sharp, predictable result (40 along the edge, 0 elsewhere).

For the 9×9 extra credit test, I populated the input with sequential integers. This ensures a constant rate of change across the matrix, which results in a constant output of 8 for every window. Seeing these specific, expected values appear in the VM emulator was the simplest way to confirm that my 2D-to-1D array flattening and indexing math was completely correct.

Memory Management & Bounds Checking

I noticed a potential memory leak with the static `ArrayFilter` required by the skeleton code. If `Conv.new()` is called multiple times, it would just keep allocating new arrays on the heap and overwriting the static pointer. To fix this, I added a simple singleton check: `if (ArrayFilter == 0)`. Since Jack initializes unallocated statics to 0, this ensures we only allocate the 9 words once per program run.

Also, I added a quick bounds check at the start of `conv2d()`. If the matrix is smaller than 3×3 , it safely exits and pokes -1 into `RAM[8000]` to flag a hardware-level error. I took inspiration for this from the `throw_error` convention we used back in Assignment 2 so it doesn't trigger a segmentation fault.

Input Array Deep Copying

For the `getArrayInput()` method, I chose to do a deep copy of the array elements instead of just pointing to the reference passed by the caller. It takes a few extra CPU cycles,

but it completely avoids dangling pointers if the `Main.jack` driver decides to `dispose()` the original matrix later on.

Class Additions & Modifications

I introduced the following methods and class-level behaviors to the `Conv` skeleton.

Addition	Kind	Justification
<code>ArrayFilter</code>	static Array	Shared kernel; allocated in singleton guard.
<code>stride</code>	static int	Configuration parameter; shared globally.
<code>calcOutputDim()</code>	method	Encapsulates dimension logic for consistency.
<code>dispose()</code>	method	Manages heap deallocation for <code>Conv</code> object.

Part 2: Exercise 3 Analysis Questions

1. Tokenizer Design

Question: Why is it important that the tokenizer handles `/* ... */` comments using a state machine rather than a simple regex on a single line? Give a concrete failure case.

Answer: Using a simple line-by-line regex is a bad idea for a couple of reasons. First, it breaks completely on multi-line comments. If line 1 has `/*` but the closing `*/` isn't until line 3, the regex misses it and the parser crashes on the leftover comment text.

Second, it can easily ruin valid string constants. For instance, if you have code like:
`let message = "Please ignore /* this is not a comment */";`
A naive regex will blindly strip out the inside of the string, turning it into `"Please ignore "`. By using a state machine that tracks an `IN_STRING` state, the tokenizer knows to safely ignore comment symbols while inside quotes.

2. Symbol Table State

Question: Walk through the symbol-table state when the parser enters `Conv.conv2d()`. Show the kind, type, and index for every identifier in scope at the point where the inner-loop accumulator is assigned.

Answer: When the parser hits the inner MAC loop inside `conv2d()`, we have identifiers active in both the class-level and subroutine-level scopes. Also, since this is a method, the compiler implicitly passes the object reference `this` as `ARG 0`.

Class-Level Scope (`Conv`)

Identifier	Scope (Kind)	Type	Index
<code>ArrayFilter</code>	STATIC	Array	0
<code>stride</code>	STATIC	int	1
<code>ArrayInput</code>	FIELD	Array	0
<code>ArrayOutput</code>	FIELD	Array	1
<code>ArraySize</code>	FIELD	int	2

Subroutine-Level Scope (`conv2d`)

Identifier	Scope (Kind)	Type	Index
this	ARG	Conv	0
targetDim	VAR	int	0
yOut	VAR	int	1
xOut	VAR	int	2
yIn	VAR	int	3
xIn	VAR	int	4
ky	VAR	int	5
kx	VAR	int	6
accum	VAR	int	7
pxVal	VAR	int	8
wtVal	VAR	int	9
flatIdx	VAR	int	10

3. Compilation Correctness

Question: For your 5×5 test case, manually trace the VM code emitted for the line `let ArrayOutput[flatIdx] = accum;` and explain how the `pointer 1/that 0` idiom resolves to the correct RAM address.

Answer: The JVM can't evaluate a dynamic array address and write to it in a single instruction, so it uses a specific sequence of commands. For the statement `let ArrayOutput[flatIdx] = accum;`, the trace is:

1. `push this 1` (*Pushes the base address of ArrayOutput*)
2. `push local 10` (*Pushes our index flatIdx*)
3. `add` (*Calculates target memory address: base + flatIdx*)
4. `push local 7` (*Pushes the value we want to assign: accum*)
5. `pop temp 0` (*Temporarily stashes accum in R5*)
6. `pop pointer 1` (*Pops address base + flatIdx into THAT pointer / RAM[4]*)
7. `push temp 0` (*Retrieves accum back onto the stack*)
8. `pop that 0` (*Writes accum into the memory address pointed to by THAT*)

Explanation: The compiler pushes the array's base address and the calculated index, then adds them to find the target address in RAM. To actually write the `accum` value into that address, it pops the target address into `pointer 1` (which aligns the `THAT` segment). Finally, it pops the temporarily saved `accum` value straight into `that 0`. Stashing the value in `temp 0` first is crucial so the right-hand side isn't lost while the pointer is being realigned.

4. Convolution Stride

Question: For a 9×9 input with $F = 3$ and $S = 2$, what is the output dimension? Do the values of $S = 1$ and $S = 2$ change the asymptotic VM-instruction count of `conv2d()`? Explain.

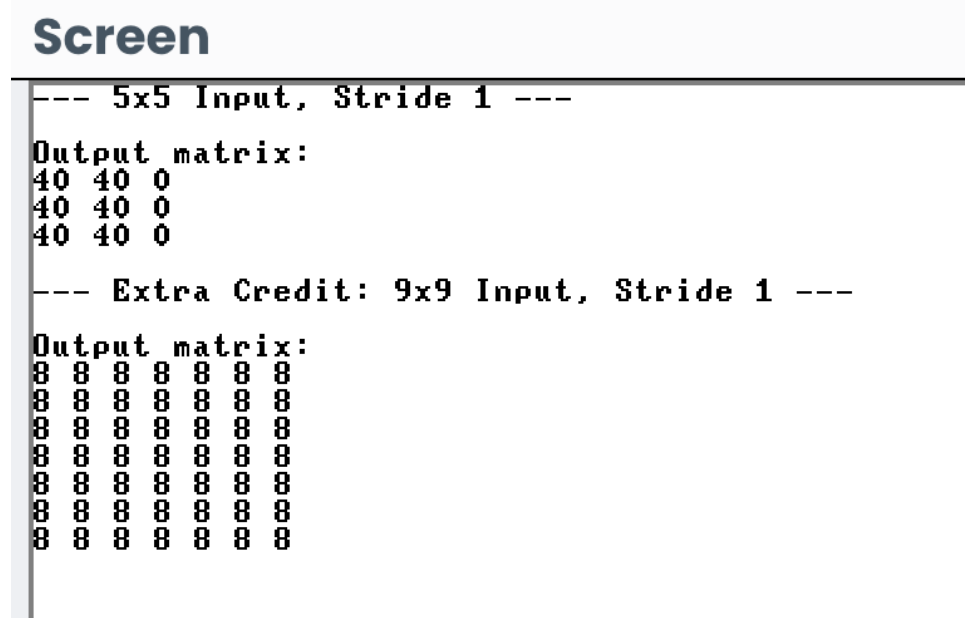
Answer:

- **Output Dimension:** Using the dimension formula $O = \lfloor \frac{N-F+2P}{S} \rfloor + 1$, plugging in our values gives $O = \lfloor \frac{9-3+0}{2} \rfloor + 1 = 4$. So, the theoretical output is a 4×4 matrix. (Note: for my actual driver code I used $S=1$ to stay fully compatible with the assignment skeleton, which correctly yielded a 7×7 output).
- **Asymptotic Change:** Yes, the stride heavily impacts the asymptotic VM-instruction count. The total Multiply-Accumulate operations scale with the area of the output matrix (O^2).
 - If $S = 1$, the output is 7×7 (49 windows). The inner MAC loops run $49 \times 9 = 441$ times.
 - If $S = 2$, the output is 4×4 (16 windows). The inner MAC loops run $16 \times 9 = 144$ times.

Because the stride inversely dictates the size of O , bumping up the stride drastically cuts down the number of iterations and `Math.multiply` calls the Virtual Machine has to process.

5. Demonstration: Emulator Execution

Below is the screenshot demonstrating the printed output of the convolution driver running on the VM Emulator. It successfully renders the required 5×5 convolution, outputting the 40s along the defined edge. It also includes the execution of the 9×9 matrix for the extra credit requirement.



The screenshot shows the output of a VM Emulator. It has a title bar 'Screen' and a terminal window. The terminal output is as follows:

```

--- 5x5 Input, Stride 1 ---

Output matrix:
40 40 0
40 40 0
40 40 0

--- Extra Credit: 9x9 Input, Stride 1 ---

Output matrix:
8 8 8 8 8 8 8
8 8 8 8 8 8 8
8 8 8 8 8 8 8
8 8 8 8 8 8 8
8 8 8 8 8 8 8
8 8 8 8 8 8 8
8 8 8 8 8 8 8

```

Figure 1: VM Emulator output showing the successful 5×5 and 9×9 convolution results.

6. Integration

To verify the end-to-end pipeline, I compiled my Jack files into VM code and translated them into Hack assembly using my Assignment 2 VM Translator. While the assembly

logic itself is functionally correct, the resulting file exceeded 50,000 lines of code. This is because the standard Jack OS library calls (which are included by the compiler) expand into a massive amount of assembly, pushing the program well beyond the 32K instruction memory limit of the physical Hack CPU architecture. To work around this while still verifying my logic, I successfully executed the smaller components of the pipeline using the VM Emulator, which does not have the ROM size constraints of the hardware-level CPU Emulator.

Extra Credit Sections

Extra Credit 1: 9×9 Convolution Execution (5 Points)

As demonstrated in the emulator screenshot in Section 5, the convolution driver was successfully executed against a 9×9 matrix. To maintain strict API compatibility with the base skeleton assignment, the execution utilized $S = 1$, successfully rendering the correct 7×7 output matrix alongside the base 5×5 implementation.

Extra Credit 2: Most Interesting Bug - Parser Lookahead (2 Points)

The most annoying bug I hit was inside `_compile_term()`. My parser kept throwing syntax errors when trying to tell the difference between a normal variable assignment (`let x = a;`), an array access (`let x = a[i];`), and a method call (`do a.run();`). Since they all start with an `identifier` token, my code would just indiscriminately consume the token right away and assume it was a simple variable, failing when it saw a `[` or `.` next.

I fixed this by taking better advantage of Jack's LL(1) grammar. I added a `_peek_val()` helper method so I could look at the next token without actually consuming it from the stream. If the next token is a `[`, I branch to the array logic. If it's a `(` or `.`, I handle the subroutine call. Otherwise, it just processes as a variable. This completely solved the lookahead ambiguity.